

Using Fabric test network for Two Orgs

Section 1) Prerequisite Installation:

Assuming you already have git installed on your computer. Check the git version using the command provided below. If you do not get any version in the log, you need to install git:

```
git --version
```

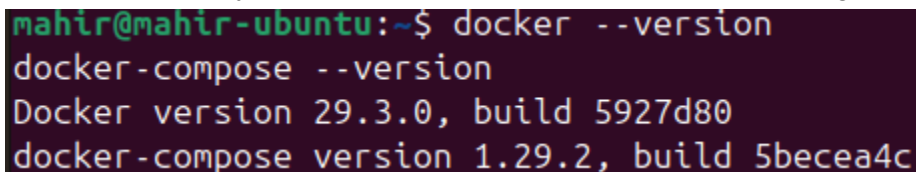
Install the latest version of **git**, **cURL**, and **Docker** if they are not already installed:

```
sudo apt-get update
sudo apt-get install git
sudo apt-get install curl
sudo apt-get -y install docker-compose
sudo apt-get install jq
```

Once installed, confirm that the latest versions of both **Docker** and **Docker Compose** executables were installed. Because Fabric nodes run in isolated environments, we heavily rely on Docker.

```
docker --version
docker-compose --version
```

This should show you an output in the terminal similar to the image below:

A terminal window with a dark background and light-colored text. The prompt is 'mahir@mahir-ubuntu:~\$'. The user has entered 'docker --version' and 'docker-compose --version'. The output shows 'Docker version 29.3.0, build 5927d80' and 'docker-compose version 1.29.2, build 5becea4c'.

```
mahir@mahir-ubuntu:~$ docker --version
docker-compose --version
Docker version 29.3.0, build 5927d80
docker-compose version 1.29.2, build 5becea4c
```

Make sure the **Docker daemon** is running. For that, run the following code:

```
sudo systemctl start docker
```

In Linux, Docker usually requires "root" (**sudo**) privileges. However, **using **sudo** in Hyperledger Fabric will permanently corrupt your cryptographic files.** We must add your user to the Docker group so you no longer need to use **sudo** with Docker.

```
sudo usermod -a -G docker $USER
newgrp docker
```

CLEANUP STEP (Fixing Broken PCs):

If this lab has ever been attempted on this PC before, there are likely hidden Docker volumes and root-locked files that will break your channel creation.

Run these commands to completely wipe the slate clean before starting.

Note: If any of these commands say "requires at least 1 argument" or "No such file", it just means your PC is already clean.

```
# 1. Navigate to the home directory
cd ~

# 2. Force-delete the entire old lab folder (if it exists)
sudo rm -rf ~/fabric-lab

# 3. Kill all running or hidden Docker containers
docker rm -f $(docker ps -aq) 2>/dev/null

# 4. Destroy all orphaned virtual hard drives (Volumes)
docker volume rm $(docker volume ls -q) 2>/dev/null

# 5. Do a final deep clean of the Docker engine
docker system prune -af --volumes
```

Section 2) Starting to Install Fabric:

Now, create a directory where we will do our lab. In this case, we have named our directory or folder **fabric-lab**

```
cd ~
mkdir fabric-lab
cd fabric-lab
```

Now, by running the following Curl command, we download the necessary files to **install Fabric**, make them executable, and finally, with the install command, we will install Fabric.

```
curl -sSLO
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh
```

It might take a bit of time, depending on the network speed. It will create a fabric-samples folder.

We need to go to the fabric-samples folder and then the bin folder inside it to know some info.

```
cd fabric-samples/bin
```

Check the peer and orderer binaries version.

Peer: A computer/node that holds a copy of the ledger database and executes the Chaincodes.

Orderer: There is no "mining" in Fabric; the Orderer node simply receives transactions, sequences them into a specific order, packages them into a block, and sends them to the Peers to save.

```
./peer version  
./orderer version
```

Match the versions from here (Peer version: 2.5.15 and Orderer version 2.5.15) :

```
mahir@mahir-ubuntu:~/fabric-lab/fabric-samples/bin$ ./peer version  
./orderer version  
peer:  
  Version: v2.5.15  
  Commit SHA: 83c7930  
  Go version: go1.26.0  
  OS/Arch: linux/amd64  
  Chaincode:  
    Base Docker Label: org.hyperledger.fabric  
    Docker Namespace: hyperledger  
  
orderer:  
  Version: v2.5.15  
  Commit SHA: 83c7930  
  Go version: go1.26.0  
  OS/Arch: linux/amd64
```

Checkpoint - 1 (Show the instructor the above part in your terminal)

Section - 3) Running the Test-Network and deploying the Chaincode:

Go to the **test network** folder in your working directory, where **we will continue the entire lab, and then, for a safety check, disconnect** the network again.

```
cd ~/fabric-lab/fabric-samples/test-network
```

Now, we will start the main part of your lab:

Bring up the network and the Channel creation:

Because Fabric is private, a Certificate Authority. We are using the **-ca flag** to boot up three CAs (One for Org1, one for Org2, and one for the Orderer). They will generate the cryptographic keys needed for the network to trust each other.

```
./network.sh up -ca
```

You can check the running containers anytime with the following command (basically, you can see **running peers, orderers, and CAs running in Docker**):

```
docker ps
```

Now we will create a Fabric channel. In our case, this channel is named **“mychannel”**. A channel is a private **“Subnet”** or chat room between **specific organizations**. If Org1 and Org2 are on "Channel A", Org3 cannot see any data or transactions happening on Channel A.

```
./network.sh createChannel -c mychannel -ca
```

By scrolling through the logs, you should see a long output in the terminal, where the following screenshot shows 2 Orgs joining the Created channel named **“mychannel”**. (Our test network **already has two orgs set up on it**) should be printed too.

```
Creating peer0.org1.example.com ... done
Creating orderer.example.com ... done
Creating peer0.org2.example.com ... done
```

Let's now **deploy the chaincode** pre-written with JavaScript via the Node Environment. The following command creates a **Fabric NodeJS chaincode environment**.

```
docker pull hyperledger/fabric-nodeenv:2.5
```

Deploying the chaincode on the Channel:

Here, the chaincode name is **“basic”**. This chaincode, to be deployed by **the deployCC** part, is installed into both organizations and then deployed on the channel.

```
./network.sh deployCC -ccn basic -ccp
../asset-transfer-basic/chaincode-javascript -ccl javascript
```

```
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1M
SP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1M
SP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
```

You should see the above part in your logs, where the chaincode has been committed for both orgs on the channel.

Checkpoint - 2 (Show the instructor the above part in your terminal that the chaincode has been successfully deployed.)

We will now act as **Org 1** to put data on the blockchain. Make sure you are still in the **test-network** folder:

Section - 4) Interacting with the Network:

After deploying the chaincode, you have to **invoke** it. For invocation, we need to first access the MSP certificates.

In the Fabric terminal, we don't have MetaMask. By copying these export commands, we are telling the terminal exactly where our Org1 cryptographic "ID Badge" (certificates) are located, so we can digitally sign our commands as Org1.

```
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=$PWD/../config/
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

Here, local host: **7051 is a peer which is Org 1** in this case.

```
peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
"${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 --tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \
--peerAddresses localhost:9051 --tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
-c '{"function": "InitLedger", "Args": []}'
```

Here, the local host: **7050** connects the orderer to this network.

```
mahir@mahir-ubuntu:~/fabric-lab/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 --tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \
--peerAddresses localhost:9051 --tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
-c '{"function":"InitLedger","Args":[]}'
2025-08-28 21:54:43.124 +06 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

This query shows all channel assets on the network.

```
mahir@mahir-ubuntu:~/fabric-lab/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Notice we use *invoke* when we want to write or modify data on the blockchain.

Notice we use *query* when we just want to read data without modifying the state.

Let's invoke the chaincode for a Peer Org; for example, **transfer asset6 to Christopher**

```
peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 --tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt" \
--peerAddresses localhost:9051 --tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt" \
-c '{"function":"TransferAsset","Args":["asset6","Christopher"]}'
```

```

mahir@mahir-ubuntu:~/fabric-lab/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/
/tlsca.example.com-cert.pem" \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 --tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org1.example.com/
peers/peer0.org1.example.com/tls/ca.crt" \
--peerAddresses localhost:9051 --tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org2.example.com/
peers/peer0.org2.example.com/tls/ca.crt" \
-c '{"function":"TransferAsset","Args":["asset6","Christopher"]}'
2025-08-20 22:03:53.323 +06 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful.
result: status:200 payload:"Michel"

```

After running the following query, we can see that the owner of asset6 is now Christopher.

```

peer chaincode query -C mychannel -n basic -c
'{"Args":["ReadAsset","asset6"]}'

```

```

mahir@mahir-ubuntu:~/fabric-lab/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic
-c '{"Args":["ReadAsset","asset6"]}'
{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Christopher","Size":15,"docType":"asset"}

```

Without closing this terminal, open another terminal and do the same steps, so that we use Org2 on the same channel.

In the new terminal, navigate to the test-network folder:

```

cd ~/fabric-lab/fabric-samples/test-network

```

Configure Terminal 2 with the "ID Badge" for Org 2:

```

export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=$PWD/../config/
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org2MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org2.example.com/users/Admin@org2.example.com/msp
export CORE_PEER_ADDRESS=localhost:9051

```

Query the ledger to see if Org 2 recognizes the transfer made by Org 1:

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["ReadAsset","asset6"]}'
```

```
mahir@mahir-ubuntu:~/fabric-lab/fabric-samples/test-network$ peer chaincode query -C  
mychannel -n basic -c '{"Args":["ReadAsset","asset6"]}'  
{\"AppraisedValue\":800,\"Color\":\"white\",\"ID\":\"asset6\",\"Owner\":\"Christopher\",\"Size\":15,  
\"docType\":\"asset\"}
```

You should see that asset6 is owned by Christopher, confirming that both organizations share the exact same World State.

(The main issue here is that the channel and the CLI are not dynamic. Think about how that can be made possible by looking at the file contents in your future time.)

Checkpoint - 3 (Show to the instructor this part in your terminal that the asset was transferred in both terminals for each Org.)

Section 5) Viewing the Identity Structure

Let's look at what the "ID Badges" actually look like on your computer. We will use the `tree` tool.

```
sudo apt update  
sudo apt install tree
```

Following the next command will let us see the structure clearly:

```
tree  
organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.c  
om/
```

You will see a **signcerts** folder (This is your **Public Key**, which you share with the network) and a **keystore** folder (This is your **Private Key**, which is used to sign transactions).

```
mahir@mahir-ubuntu:~/fabric-lab/fabric-samples/test-network$ tree organizations/peer  
Organizations/org1.example.com/users/Admin@org1.example.com/  
organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/  
├── msp  
│   ├── cacerts  
│   │   └── localhost-7054-ca-org1.pem  
│   ├── config.yaml  
│   ├── IssuerPublicKey  
│   ├── IssuerRevocationPublicKey  
│   ├── keystore  
│   │   └── 2b5f795ca826d695919626ac23a4f45e486a690f09eb87522f473485683bba52_sk  
│   ├── signcerts  
│   │   └── cert.pem  
│   └── user  
6 directories, 6 files
```

Checkpoint - 4 (Show the instructor this part in your terminal, how the MSP structure looks.)

You must always shut down your network when you are finished. If you just close the terminal window, the Docker containers will run forever and break your computer for the next lab.

```
sudo ./network.sh down
```

- Source: https://hyperledger-fabric.readthedocs.io/en/release-2.5/test_network.html